

GymVerse Gym Management System

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in the Department of Computer Science and Engineering

II Year, II Semester

Submitted by

2510030006: N Srikar

2510030008: G Pranav

2510030147: Yogendra

2510030246: Sri Charan

2510030306: K Abhineshwar

Under the guidance of

P Lalitha



Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

Academic Year 2026-2027

Abstract

GymVerse is a web based gym management system that brings membership administration, attendance, trainer assignments, workouts and class bookings into a common application. We developed the project to connect routine gym operations to a structured database, so that staff and members can work with consistent records rather than separate registers or spreadsheets.

The application uses a React frontend and an Express backend. PostgreSQL stores accounts and operational records, with Neon serving as the hosted database and Render as the backend hosting platform. The design separates membership plans from individual subscriptions, scheduled classes from bookings, and exercise definitions from assigned workout plans. Foreign keys, unique constraints and transactions protect relationships between these records. Role checks and record ownership checks restrict access for administrators, receptionists, trainers and members.

The public website presents training information, membership plans and upcoming classes. Authenticated pages support the operational workflows. An optional assistant answers questions using the plan catalogue; optional MongoDB storage retains chat conversations without becoming the main gym database. Password recovery and email confirmation depend on an email service configuration.

Recorded project verification completed 71 full-stack checks, 16 frontend tests and three development-origin tests, and produced a successful frontend build. These results demonstrate the tested local flows; they do not establish production capacity or availability. The project illustrates how relational modelling, API validation and a role-aware interface can support a working gym administration application. Further work includes production monitoring, broader concurrency testing and payment gateway integration.

Keywords: gym management, PostgreSQL, React, Express, role based access, database transactions.

List of Figures

Figure 1 GymVerse application architecture	7
Figure 2 Core membership and class relationships.....	8

List of Tables

Table 1 User roles and responsibilities..... 6

Table 2 Core database entities..... 8

Table 3 Functional verification scenarios..... 11

Table 4 Recorded verification results..... 12

Table of Contents

Abstract.....	2
List of Figures.....	3
List of Tables	4
I Introduction	6
II Methodology	7
Architecture and technology choices	7
Database design	8
Integrity and security	9
Implementation and deployment.....	10
III Experiments	11
IV Results	12
V Conclusion and Future Work	13
References.....	14

GymVerse Gym Management System

I. INTRODUCTION

A gym manages several related activities each day: registering members, assigning subscriptions, recording payments, tracking attendance and coordinating training. When these activities are maintained in separate records, staff must repeatedly compare information. A payment may be recorded without an updated subscription, or a class list may not reflect its remaining capacity. These inconsistencies motivate a system in which operations share a common database.

Our objective is to provide one application for these activities while retaining clear responsibility for each user. GymVerse separates the public website from the authenticated workspace. Visitors can explore membership plans and upcoming classes, while signed-in users receive pages appropriate to their role. The backend validates requests independently of the interface, and PostgreSQL enforces structural rules that remain valid even when more than one request reaches the system.

The scope includes members, trainers, membership plans, subscriptions, payment records, attendance, exercises, workout plans, workout sessions and class bookings. Dashboards and reports summarize the stored operational data. Optional chat assistance and saved conversations extend the interface. Payment records describe transactions entered into the application; they do not by themselves implement online payment processing or bank settlement.

Table 1 User roles and responsibilities

Role	Main responsibility
Administrator	Manage classes, plans, staff approvals, users and reports.
Receptionist	Manage member records, subscriptions, payments and attendance.
Trainer	Work with assigned members and training plans within permitted access.
Member	View permitted personal information, training and class bookings.

The project applies database concepts to a practical workflow. It demonstrates primary and foreign keys, constrained status values, transactions, indexes, SQL aggregation, audit records and schema migrations. Its success is assessed through functional checks and consistency rules rather than an assumed improvement in gym revenue or member fitness. Those outcomes would require a separate study with real operational data.

II. METHODOLOGY

A. Architecture and technology choices

We organized GymVerse as a three-tier application. React renders the browser interface and manages navigation. Axios sends JSON requests to the Express API. The API applies authentication, authorization and validation before issuing parameterized SQL through the PostgreSQL driver. This structure keeps presentation decisions separate from database operations and gives all clients the same server-side rules [1, 2].

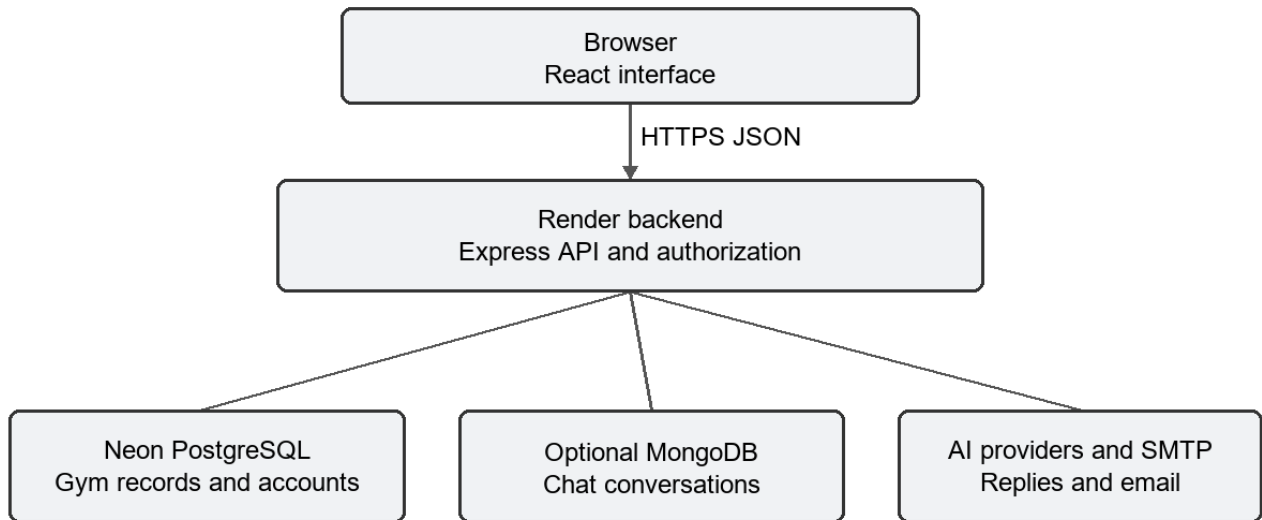


Figure 1 GymVerse application architecture

PostgreSQL is the system of record for gym operations. Neon provides the hosted PostgreSQL database, while Render runs the backend. The frontend can be hosted separately or served from the backend after a production build. Optional services have narrower roles: MongoDB stores chat conversations, AI providers generate assistant replies, and SMTP delivers account emails. A failure in optional chat storage is handled with temporary chat behavior.

The development process follows the data model. We define entities and relationships, expose controlled API operations, connect the user interface, and verify the resulting flows. The SQL files remain explicit rather than hidden behind an object-relational mapper. Numbered migrations record applied changes so an existing database can receive upgrades without reloading demonstration data [3].

React Router organizes public and protected pages. Shared interface components provide forms, tables, pagination and feedback states. Lazy loading divides the authenticated application into smaller modules. The visual design uses a dark background, lime accents and gym photography, while membership prices and class availability come from API responses rather than being fixed in the page.

B. Database design

The schema separates reusable definitions from individual events. A membership plan defines a product; a subscription records a member enrolling in that product for a period. A fitness class defines a session type; a class schedule places it on a date with a trainer and capacity. Class bookings connect members to scheduled sessions [3].

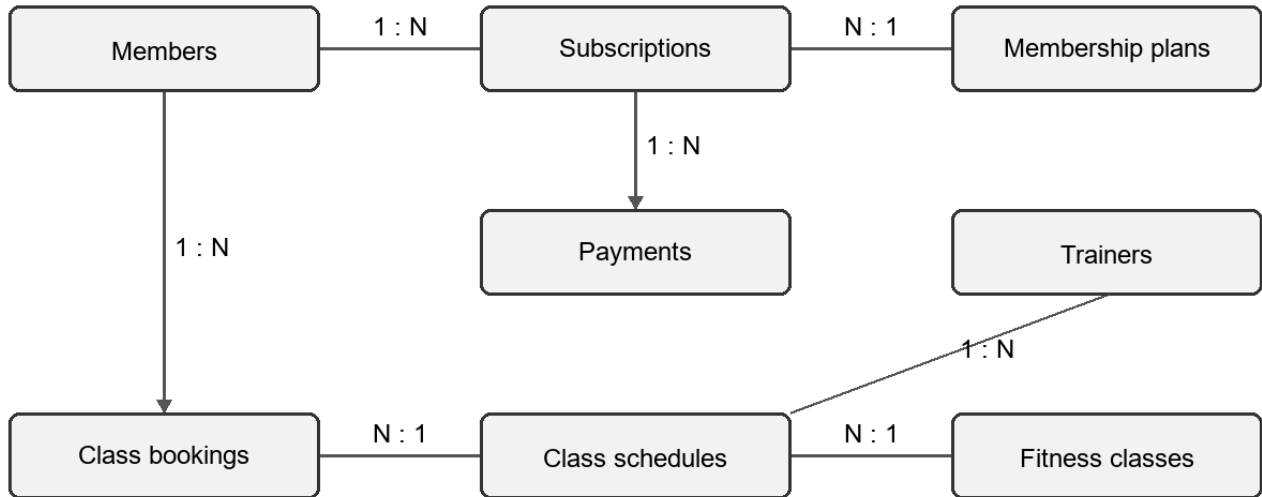


Figure 2 Core membership and class relationships

Table 2 Core database entities

Entity group	Stored information
roles and users	Login identities, role links and account state.
members and trainers	Profiles and separate links to login accounts.
membership_plans and subscriptions	Plan definitions and time-bound member enrolment.
payments and attendance	Financial records and member visits.
exercises and workout_plans	Reusable exercises and ordered training prescriptions.
member_workouts and workout_sessions	Assignments and recorded training sessions.
fitness_classes, class_schedules, class_bookings	Class definitions, timetable and reservations.
audit_logs and schema_migrations	Change history and applied schema versions.

Identity primary keys give each record a stable identifier. Foreign keys prevent orphaned references and define deletion behavior. Associative tables handle relationships such as workout plans containing many exercises and members booking many schedules. This separation reduces repeated descriptive data. The diagram shows the central membership and booking relationships rather than every table in the repository.

C. Integrity and security

The database and API enforce complementary rules. Required fields reject incomplete records, unique values prevent duplicate identities, and check constraints restrict amounts, capacities and status values. The active-subscription partial index permits historical subscriptions while preventing more than one active subscription for a member. API validation adds readable feedback before the database rejects invalid input [3, 4].

Subscription creation uses a transaction. The controller locks the member record, checks the member and selected plan, creates the subscription and opening payment, and commits the related changes together. If an operation fails, rollback prevents a partially completed enrolment. Locking the member row also serializes competing requests for the same member, including when no active subscription row exists yet.

Class booking uses a row lock on the schedule. Within the transaction, the controller checks the session, current bookings and available capacity before reserving a place. Requests for the same class therefore inspect capacity in a controlled order. The locking mechanism is implemented in the controller; a dedicated concurrent load test remains necessary to measure behavior under sustained contention.

Passwords are hashed with bcrypt, and authenticated requests carry a JSON Web Token. Route-level permissions distinguish staff and member operations. Record-level checks then verify ownership or an allowed trainer relationship. This second check matters because a valid member token must not grant access to another member merely by changing an identifier in a URL.

Account recovery uses time-limited tokens, while session invalidation is supported through account token versions and revoked-token records. Request rate limits constrain repeated authentication and chat calls. Parameterized SQL separates data from query syntax. Audit handling removes password hashes before storing change snapshots, and production error responses avoid returning internal stack traces.

These controls address specific risks and do not constitute a complete security certification. Operational protection also requires confidential environment variables, restricted database access, backups and maintained dependencies. The browser currently stores its login token in local storage, so preventing script injection remains especially important. A future authentication review can evaluate HTTP-only cookies and the additional cross-site request protections they require.

D. Implementation and deployment

The landing page introduces the gym, training programs and facility experience. It reads the plan catalogue and upcoming schedules from public endpoints. Loading, error and empty states keep the interface understandable when data is delayed or no classes have been scheduled. After login, the dashboard and navigation expose role-appropriate actions. The API remains responsible for authorization even when a button is hidden in the browser [2, 4].

The backend groups routes by resource, including authentication, members, trainers, membership plans, subscriptions, payments, attendance, workouts and schedules. Controllers carry out validation-dependent operations, and shared helpers manage transactions, pagination and errors. Dashboards use SQL aggregation so the browser receives summaries rather than downloading all operational records.

The assistant is available to signed-in users and reads the current plan catalogue for membership questions. Provider fallback and time limits handle unavailable AI responses. MongoDB chat history is optional and isolated from the main operational database. The application must not treat assistant output as a replacement for authorized database actions or professional advice.

For the hosted setup, Neon supplies PostgreSQL connection settings and Render runs the Express process. The repository currently reads individual database environment variables. The frontend API address, backend allowed origin and account-link base URL must refer to the deployed domains. A production frontend build is required when its API address changes because Vite embeds that setting during compilation.

Schema upgrades use the migration command, which applies pending numbered SQL files and records their checksums. The required-role migration inserts the four application roles without changing existing role identifiers. Development seed data is separate and must not be replayed over an existing production database. The newer demo timetable provides relative future dates for fresh development installations; administrators create actual classes in the deployed application.

A release check should confirm that the health endpoint reaches PostgreSQL, public plans and schedules load, login succeeds, and protected operations respect user roles. Publishing a backend revision alone does not publish a separately hosted frontend. Production deployment verification and backup restoration exercises should therefore be treated as separate operational checks, not inferred from a successful local build.

III. EXPERIMENTS

Verification is organized into frontend tests, backend regression tests and a full-stack smoke check. Frontend tests exercise application behavior with Vitest. Backend tests use Jest and request-level helpers to check validation and authorization. The smoke script creates an isolated PostgreSQL database, runs migrations and seed data, starts the API, performs checks and removes that test database [5].

Table 3 Functional verification scenarios

Scenario	Expected behavior
Correct and incorrect login	Accept valid credentials; reject invalid credentials with a usable message.
Member registration	Create a member account and its related profile.
Cross-member access	Reject attempts to read or modify another member's protected records.
Trainer directory privacy	Withhold contact details from members while retaining permitted staff access.
Pagination validation	Accept valid pages and reject negative or excessive values.
Migrations applied twice	Second execution leaves already applied migrations unchanged.
Dashboard revenue	Return numeric values in chronological month order.
Audit inspection	Do not retain password hashes in logged snapshots.
Development browser origins	Accept the configured localhost origin and its loopback alias.

The recorded smoke run covered authentication, main read endpoints, ownership regressions, indexes and selected write behavior. Its output provides a repeatable functional baseline. It is not a comprehensive benchmark: there are no measured throughput, latency percentile or production availability results in this report.

Further experiments should run simultaneous reservations for the last class place, subscription races, provider timeouts and expired-session flows. These tests should use disposable databases and controlled accounts. Live Neon records should remain outside any workflow that drops or recreates a database. Email delivery and hosted domain configuration also require environment-specific checks.

IV. RESULTS

The recorded verification established that the tested local build could initialize its database, start the API and serve the main application workflows. The frontend build also completed. Table 4 reports the observed counts from that verification session; these are recorded results, not a claim that the current hosted deployment was retested [5].

Table 4 Recorded verification results

Verification	Observed outcome	Scope
Full-stack smoke check	71 passed, 0 failed	Fresh database, migrations and API checks.
Frontend test suite	16 passed	Three frontend test files.
Development-origin tests	3 passed	Loopback aliases and production restrictions.
Frontend production build	Succeeded	Compiled deployable frontend assets.
HTTP origin checks	Both returned HTTP 200	Health requests from localhost and 127.0.0.1 origins.

One integration fault was identified during the redesign: the development API accepted localhost but rejected the equivalent 127.0.0.1 browser origin. The origin helper now expands these two loopback names in development while preserving exact production origins. Direct HTTP checks confirmed a successful health response for both addresses.

The timetable also exposed a data issue. Existing demonstration schedules were historical, so the public page correctly had no upcoming entries. Fresh seed data now includes five future sessions using dates relative to installation. Existing databases are not automatically populated with these demonstrations. This distinction prevents a data-loading change from being mistaken for a schema upgrade.

The report supports functional conclusions for the exercised paths. It does not measure usability with gym staff, real payment settlement, long-running production stability or high-volume concurrent use. Optional email, AI and saved chat behavior depend on service credentials and availability. The complete backend suite, including the in-memory MongoDB history tests, is not represented by the three focused origin tests reported here.

V. CONCLUSION AND FUTURE WORK

GymVerse demonstrates a database-centered approach to gym administration. We connected a role-aware React interface to an Express API and a relational PostgreSQL model. Memberships, payments, attendance, training and bookings share consistent identifiers and relationships. Transactions protect related operations, while database constraints and authorization checks reduce inconsistent updates and unauthorized access.

The principal outcome is an implemented application with a repeatable local verification process. The recorded checks passed for the exercised flows, and the frontend produced a production build. The deployment design uses Neon for PostgreSQL and Render for the backend. Separate configuration and live verification are still required whenever the hosted system is updated.

A. Future work

The next priority is broader operational testing. Concurrent booking and subscription tests should validate behavior under contention, and monitoring should expose failed requests, database connection failures and service health. Backup restoration should be tested periodically rather than assuming a stored backup is usable.

Payment gateway integration would allow online checkout, but it requires server-side payment verification, idempotent webhook handling and reconciliation with subscription state. The current payment records provide a foundation for that work without implying that external settlement is already automated.

Additional improvements include class reminders, attendance device integrations, richer member progress views and accessibility testing across keyboard, screen reader and mobile use. The schema already contains several supporting record types, but a database field or table alone does not demonstrate an implemented interface or device integration.

The project also offers scope for a focused security review of token storage, session expiry and account recovery. These improvements should be evaluated against clear acceptance tests. Expanding the feature set is useful only when the resulting operations remain understandable to users and consistent in the database.

REFERENCES

- [1] GymVerse project repository. PROJECT_GUIDE.md. Architecture, modules, request lifecycle and database design.
- [2] GymVerse frontend. src/App.jsx, src/pages/public/Landing.jsx and src/services/api.js. Routing, public data and API integration.
- [3] GymVerse database. database/01_schema.sql through database/18_required_roles.sql; gymverse-backend/src/db/migrations.js. Schema, integrity rules, indexes and migrations. Files 07_seed.sql and 08_queries.sql contain development data and example queries rather than schema migrations.
- [4] GymVerse backend. src/app.js, src/controllers/subscriptionController.js, src/controllers/scheduleController.js and authentication middleware. API controls, transactions and booking logic.
- [5] GymVerse verification. gymverse-backend/verify_stack.js, tests/corsOrigins.test.js and the frontend Vitest suite. Recorded execution results from the project development session.
- [6] GymVerse backend and frontend README files and environment examples. Local setup, configuration, scripts and optional service boundaries.